

KAN in Recurrent Recommenders

Project plan: fix the current report, build KAN-GRU4Rec, extend to next-basket data
Saksham Agrawal, Abhishek Yadav, Yash Dagar | Guide: Dr. Amar Jindal | October 2026

Why: the current report compares GRU vs Pure KAN vs Hybrid on a random 50k-session YooChoose sample. KAN models lose on every metric and are 7 to 11x slower, but the numbers are also affected by data leakage, a broken B-spline, and copy errors. We fix those, implement GRU4Rec correctly with KAN layers, add the Instacart and Dunnhumby next-basket datasets, and finish with a new LaTeX report and one Jupyter notebook.

Deliverables: `~/Documents/kan-rec/kan_rec.ipynb` (single notebook, runs on Mac, UPES HPC, or Colab) and `report/main.tex` (same template, all result tables auto-generated by the notebook).

Phase 1: Fix the bugs in the current work

Most of these change the numbers, so the old results must be re-run, not just re-typed.

- **Data leakage:** sliding-window samples were shuffled and split 80/20 at sample level, so prefixes of the same session sit in both train and test. Fix: split by session and by time.
- **Test used as validation:** per-epoch metrics come from the test set. Fix: separate train / validation / test; early stop on validation, report test once.
- **Padding:** right padding plus reading the last output position means short sessions are encoded after pad tokens. Fix: gather the hidden state at each session's true length.
- **Broken B-spline:** the code is not the Cox-de Boor recursion in the report (same alpha used for both terms, no extended grid, dead coefficients).
- **Pure KAN cell has no gates** and runs in a Python time loop: this explains both the poor accuracy and the 10x slowdown.
- **Report errors:**
 - Recall@20 row copies HR@10; correct values are 0.6542 / 0.5850 / 0.6071.
 - Val-loss deltas have the wrong sign (+0.2829 and +0.1525, i.e. worse).
 - Says 7/7 metrics but lists 8; Hybrid time is 62.5% of KAN, not 67%; KAN crosses HR@10 = 0.50 at epoch 12, not 14.
 - Captions say KAN keeps improving, but the tables plateau by epoch 16.
 - GRU candidate equation is wrong; a KAN has one spline per edge, not per input; KAN has more parameters, so 'parameter efficient' and 'lightweight' are not supported.
 - Dataset counts do not reconcile; the caption says 'YooChoose 1/64' but sampling was random.
 - Hou et al. reference: author is Angelos Stefanidis and v8 is a critique, not a survey; Rendle 2010 is never cited.

Output: an errata list that goes into the new report's appendix.

Phase 2: Setup and compute profiles

One switch at the top of the notebook picks the hardware.

- **PROFILE = "mac"** (M1 Pro, MPS, run first): small sizes, 1 seed, few epochs. Purpose is to prove the pipeline end to end and time every model.
- **PROFILE = "upes"** (Blackwell GPU): full sizes, 3 to 5 seeds, bf16, torch.compile. Sizes are set from the Mac timings.
- **PROFILE = "colab"** (T4/L4): middle sizes, 3 seeds, checkpoints saved to Google Drive every epoch so a disconnected session can resume.
- Environment: uv with torch, pandas, scipy, matplotlib, kagglehub, jupyter. Data is downloaded with the Kaggle API (accept the Instacart competition rules once).

Output: a notebook that runs the same code on all three machines.

Phase 3: Preprocessing

Standard, published protocols so our numbers are comparable to papers.

- **YooChoose (session-based)**: standard GRU4Rec / SR-GNN protocol.
 - Drop sessions of length 1 and items seen fewer than 5 times.
 - Test = last day; validation = the day before; train = most recent 1/64 (and 1/4 on UPES).
 - Check that no session appears in two splits.
- **Instacart (next-basket)**: Python port of the nextbasketpredictionGRU protocol.
 - Each user's last order is held out; users split 50/50 into validation and test.
 - Rare products are grouped within aisles (port of their R clustering); the Mac profile uses a top-K product cap instead.
 - Keep time features (day of week, hour, days since prior order).
- **Dunnhumby Complete Journey (next-basket)**: port of the same repo.
 - Products bought at least 50 times, households with at least 3 baskets.
 - Keep the *most recent* 100 baskets (their code keeps the oldest, a bug).
 - Last basket held out; households split 50/50 into validation and test.
- **tbp-next-basket**: reuse its evaluation measures (precision, recall, F1, hit rate at k) with its F-beta formula fixed. TARS itself is Python 2, so we cite its published numbers.
- Note: published Dunnhumby benchmarks (TIFU-KNN, Reality Check) use a *different* Dunnhumby dataset, so we compare only against van Maasakkers et al. and ReCANet.

Output: cached train / validation / test splits plus a dataset statistics table written for the report.

Phase 4: KAN building block

One reusable KAN layer with several basis functions, tested before use.

- **B-spline** (correct Cox-de Boor, extended grid), **RBF** (FastKAN, about 3x faster), **Chebyshev**, and **rational** (GR-KAN, UPES only).
- Unit tests: B-spline bases sum to 1 inside the grid, gradients flow, parameter counts are printed from the code.
- A matching **MLP block with the same parameter count**, so we can tell whether a gain comes from KAN or just from extra capacity.

Output: a tested KANLinear and MLP control.

Phase 5: Baselines

Strong simple baselines first; any KAN model must beat these to matter.

- **Session:** most popular, session popularity, session item-kNN, and official GRU4Rec.
- **Next-basket:** global top-frequency, personal top-frequency, last basket, TIFU-KNN, and the basket-GRU from van Maasakkers et al.
- Sanity check against published ranges: GRU4Rec Recall@20 around 0.60 or more on yoochoose1/64; basket-GRU precision at |B| around 30% (Instacart) and 13.7% (Dunnhumby).

Output: baseline numbers that match the literature.

Phase 6: KAN-GRU4Rec models (YooChoose)

Built on the official GRU4Rec PyTorch code, because unofficial reimplementations score 30 to 99% lower.

- **S0** GRU4Rec (official settings: session-parallel batches, 2048 shared negatives, cross-entropy or BPR-max).
- **S1** + KAN head: a residual KAN on the hidden state before item scoring (cheapest, most promising).
- **S2** + KAN input transform on item embeddings.
- **S3** KAN-GRU cell: gates stay normal, only the candidate state uses KAN (keeps the gating the old Pure KAN lacked).
- **S4** S1 + S2 combined; **MLP controls** for S1 and S2.
- **Old Pure KAN and Hybrid** re-run under the fixed protocol, for continuity with the first report.
- **Ablation** on the best variant: basis type (B-spline / RBF / Chebyshev / rational) and grid size (4 / 8 / 12).

Output: a full comparison table on yoochoose1/64 (and 1/4 on UPES).

Phase 7: KAN basket-GRU models (Instacart, Dunnhumby)

Same idea applied to next-basket prediction.

- **B0** basket-GRU (multi-hot basket in, GRU, sigmoid score for every product).
- **B1** + residual KAN head; **B2** linear projection then KAN on the input; **B3** KAN-GRU cell; plus MLP controls.
- The KAN never touches the huge product dimension directly, which keeps it fast.

Output: next-basket results on both datasets.

Phase 8: Evaluation

Correct metrics, honest statistics.

- **Session:** Recall@10/20, MRR@20, NDCG@10/20 over the full item ranking.
- **Next-basket:** precision and recall at |B|/2, |B|, 2|B| and average rank (comparable to van Maasakkers et al.), plus Recall@10/20, NDCG@10/20, hit rate, and a **repeat vs new item** breakdown.
- **Cost:** parameters, seconds per epoch, inference time.
- **Statistics:** mean and standard deviation over seeds; paired Wilcoxon test vs the baseline.
- **Interpretability:** plot the learned KAN curves and measure how non-linear they really are.

Output: results tables and figures written straight into the report folder.

Phase 9: Decision point and fallback

If KAN does not help, we change the headline instead of forcing it.

- Decide on **validation** results: if no KAN variant beats both GRU4Rec and its MLP control by more than one seed standard deviation, switch focus.
- **Fallback (KAN-NBR)**: in next-basket data over 97% of correct hits are repeat purchases. A small KAN over a few per-item features (time since last purchase, purchase count, decayed frequency, popularity) learns an interpretable 'repurchase curve'. It is cheap and its curves actually mean something.
- **Session fallback**: KAN inside the feed-forward block of SASRec (as in the Kolmogorov-Arnold Transformer).
- A careful negative result (KAN does not help GRU4Rec in a fair comparison) is still a valid contribution.

Output: a clear, defensible main finding.

Phase 10: Report (deliverable 1): chapter by chapter

Same UPES template and front matter as the current report; every chapter below is either kept, fixed, or new.

- **Front matter:**
 - Title page: new title without 'Lightweight' unless results support it, e.g. 'Kolmogorov-Arnold Networks in Recurrent Recommenders: Session-Based and Next-Basket Prediction'. Same names, roll numbers, guide, logo.
 - Candidate's declaration: keep, update the title.
 - Acknowledgement: keep, also credit Instacart and dunnhumby datasets and the open-source GRU4Rec code; fix 'We sincerely thanks' and 'We also thanks'.
 - Abstract and keywords: rewritten from final results (three datasets, KAN placements, fair MLP control).
 - Table of contents, list of figures, list of tables: regenerated.
- **Ch 1 Introduction**: keep background, session-based and KAN sections; add next-basket recommendation; fix the KAN advantages list (no 'parameter efficiency' claim); new objectives and problem statement.
- **Ch 2 Literature review**: expanded, see the next section for the full list, ending with a related-work summary table and a rewritten research gap.
- **Ch 3 Datasets and preprocessing**: YooChoose, Instacart, Dunnhumby. Protocols, statistics tables and EDA figures for all three (generated as separate figures, not cropped from one image).
- **Ch 4 Model architectures**: correct KAN equations (matching the code), the four basis types, GRU4Rec, basket-GRU, every KAN placement (S1 to S4, B1 to B3), MLP controls, and TikZ architecture diagrams.
- **Ch 5 Experimental setup**: compute profiles, hyperparameters per dataset, losses, train / validation / test protocol, metric definitions (fixed MRR wording), notebook structure in place of the old file table.
- **Ch 6 Results and discussion**: session results, next-basket results with repeat vs new split, basis and grid ablations, cost table, significance tests, interpretability plots, and the fair-comparison discussion.
- **Ch 7 Conclusion and future work**: rewritten from the actual findings, including the fallback outcome if used.
- **Appendices**: A: corrected KANLinear code; B: KAN-GRU cell code; C: errata listing what changed from the first draft and why; D: full per-epoch logs.
- **References**: all cited in the text, verified titles and authors.
- **Rule**: every number and table comes from files the notebook writes; compile with tectonic locally or on Overleaf.

Output: the final report PDF.

Literature review plan (Chapter 2)

Grouped into sections. **Kept** = already in the report, **fix** = kept with a correction, **new** = to add.

- **2.1 Session-based recommendation**
 - Kept: Hidasi et al. 2015 (GRU4Rec), Li et al. 2017 (NARM), Kang and McAuley 2018 (SASRec), Sun et al. 2019 (BERT4Rec).
 - New: Hidasi and Karatzoglou 2018, top-k gains (BPR-max, negative sampling), the GRU4Rec version we actually use.
 - New: Hidasi and Czapp 2023 (RecSys), third-party GRU4Rec implementations score 30 to 99% lower; justifies using the official code.
 - New: Liu et al. 2018 (STAMP) and Wu et al. 2019 (SR-GNN), the standard comparison table for yoochoose1/64.
- **2.2 Next-basket recommendation (new section)**
 - Fix: Rendle et al. 2010 (FPMC), already in the bibliography but never cited.
 - New: Yu et al. 2016 (DREAM), the first RNN for baskets.
 - New: Guidotti et al. 2019 (TARS, the tbp-next-basket repo).
 - New: Hu and He 2019 (Sets2Sets), Hu et al. 2020 (TIFU-KNN).
 - New: Ariannezhad et al. 2022 (ReCANet), repeat-aware model with Complete Journey results.
 - New: Li et al. 2023 (Next Basket Recommendation Reality Check), repeat items dominate; motivates our repeat vs new split.
 - New: van Maasakkers, Fok and Donkers 2023 (GRU for next-basket, the linked repo), our basket-GRU baseline.
- **2.3 Kolmogorov-Arnold Networks**
 - Kept: Liu et al. 2024 (KAN).
 - Fix: Hou, Ji, Zhang and Stefanidis 2025, the current version is a critical assessment (KANs 1.36 to 100x slower), not a survey; rewrite the sentence.
 - New: Yu, Yu and Wang 2024, 'KAN or MLP: A Fairer Comparison', justifies our parameter-matched MLP controls.
 - New: efficient variants: Li 2024 (FastKAN), ChebyKAN, Yang and Wang 2025 (KAT / GR-KAN), Qiu et al. 2024 (ReLU-KAN).
- **2.4 KANs for sequential and temporal data**
 - Kept: Xu, Chen and Wang 2024 (KAN for time series), Genet and Inzirillo 2024 (TKAN).
 - Expand TKAN: it keeps the LSTM gates and uses KAN only inside, which is the design our KAN-GRU cell follows.
- **2.5 KANs in recommender systems (new section)**
 - New: Park et al. 2024 (CF-KAN, collaborative filtering).
 - New: Xu et al. 2024 (FourierKAN-GCF, graph recommendation).
 - Fix: Xu, Fang, Gong et al. 2026 (KANRec), name the model and its code; kept.
- **2.6 Related-work summary table (new):** paper, year, model type, dataset, metric, limitation.
- **2.7 Research gap (rewritten):** no published work places KAN inside GRU4Rec or applies KAN to next-basket prediction, and no KAN recsys paper uses a parameter-matched MLP control. Softens the old 'no prior work on KAN recurrent cells' claim, since TKAN exists.

Notebook (deliverable 2)

- `kan_rec.ipynb`: profile switch, data download and preprocessing, KAN module with tests, baselines, all models, evaluation, plots, and export of every table and figure into `report/generated/` and `report/figures/`.

- Runs top to bottom on Mac, UPES or Colab by changing one line.

Extra ideas worth considering

- Make KAN-NBR a main contribution even if KAN-GRU4Rec works; it is the most novel and most interpretable piece.
- Feed time features (hour, day of week, days since last order) through a KAN in basket-GRU to get readable effect curves.
- Add Diginetica as a second session dataset, since it is standard in published tables.
- Try KAN grid extension during training (a KAN-specific trick from the original paper) as an ablation.

Suggested order of work

- Weeks 1 to 2: phases 1 to 4 on the Mac profile.
- Weeks 3 to 4: phases 5 to 7 smoke runs on Mac, then full runs on UPES or Colab.
- Week 5: phases 8 and 9.
- Week 6: phase 10 (literature review can be written in parallel from week 1).